

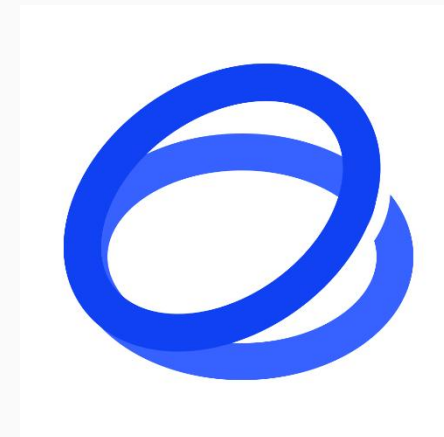
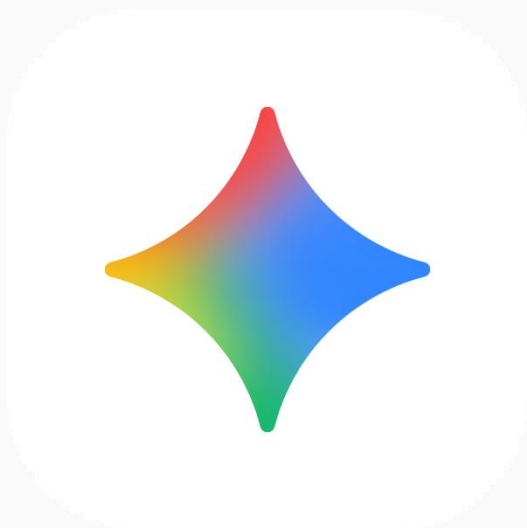
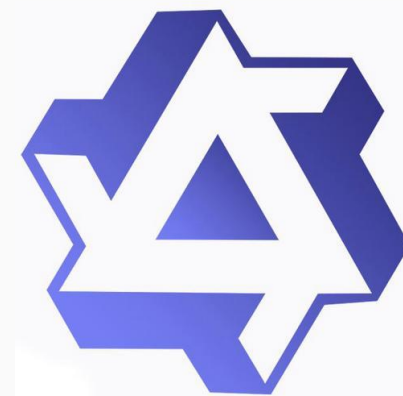
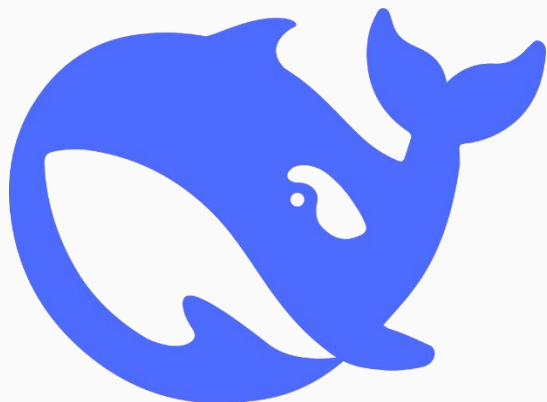
KernelGen专题：让AI自己写算子

韩冬煦

北京智源研究院

2026年08月05日

在你与 AI 聊天的时候，AI在干什么



每次回答，背后是数万亿次计算在同时发生

在算什么？算子(Kernel)

算子简介

- 算子是计算机科学和深度学习领域最基础的计算原语
- 算子是构建任何神经网络的最小可执行的功能单元

算子核心价值

- 算子将复杂的数学运算打包成一个易于管理和复用的模块
- 算子是实现深度学习模型高性能的核心载体
- 无论是底层框架开发者、硬件制造商还是AI工程师，都需要围绕算子进行高效的设计、实现和部署。

算子组成要素

- 数学逻辑：计算算子本质，算子的灵魂
- 输入输出张量：算子执行计算所需的数据载体
- 其它要素：属性、内存布局、并行策略

AI大模型的运作，就像一家高效运转的餐厅

北京智源人工智能研究院
BEIJING ACADEMY OF ARTIFICIAL INTELLIGENCE



菜单 = 模型

菜单上有很多菜(功能)，决定了你能做什么



模型越大、功能越多，
就像菜单越丰富



灶台 = 芯片

厨房的设备(灶台、烤箱等)，决定了你能做得多快、多大规模



芯片性能越强，
餐厅出餐速度越快



烹饪步骤 = 算子

每一道菜都有详细的烹饪步骤，决定了怎么做、做的好不好、快不快



算子就是这些烹饪步骤，把数学
计算拆解成一步步的具体操作

没有好的烹饪步骤，再好的食材、再贵的灶台，也做不出一道美味的菜！



算子，是连接模型与芯片的关键桥梁，
让 AI 真正**动起来**，又快又好地完成计算

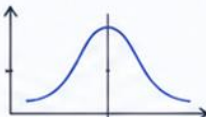


一个高性能算子工程师，需要**同时懂很多东西**

π 数学

理解算法，保证正确性

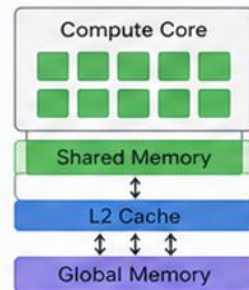
- 线性代数
- 微积分
- 概率统计
- 数值分析

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \times \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} ax + by \\ cx + dy \end{pmatrix}$$

$$\sum_{i=1}^n x_i$$

硬件架构

了解硬件，才能用好硬件

- GPU / NPU 架构
- 并行计算模型
- 内存层次结构
- 指令集与存储体系



`</>` 编译器

让代码被高效地执行

- 编译原理
- 中间表示 (IR)
- 指令调度
- 代码优化 (优化器)

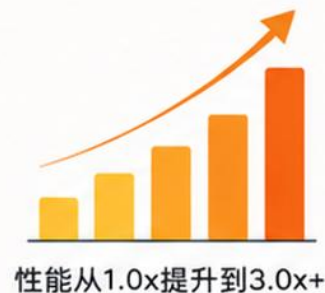


高性能算子工程师

性能优化

极致压榨每一分性能

- 算子融合
- 内存优化
- 并行优化
- 性能分析与调优



这样的人才，**非常稀缺!**

全栈能力 + 深入经验 + 大量实践
缺一不可，培养周期长，行业供给远远不足



稀缺!



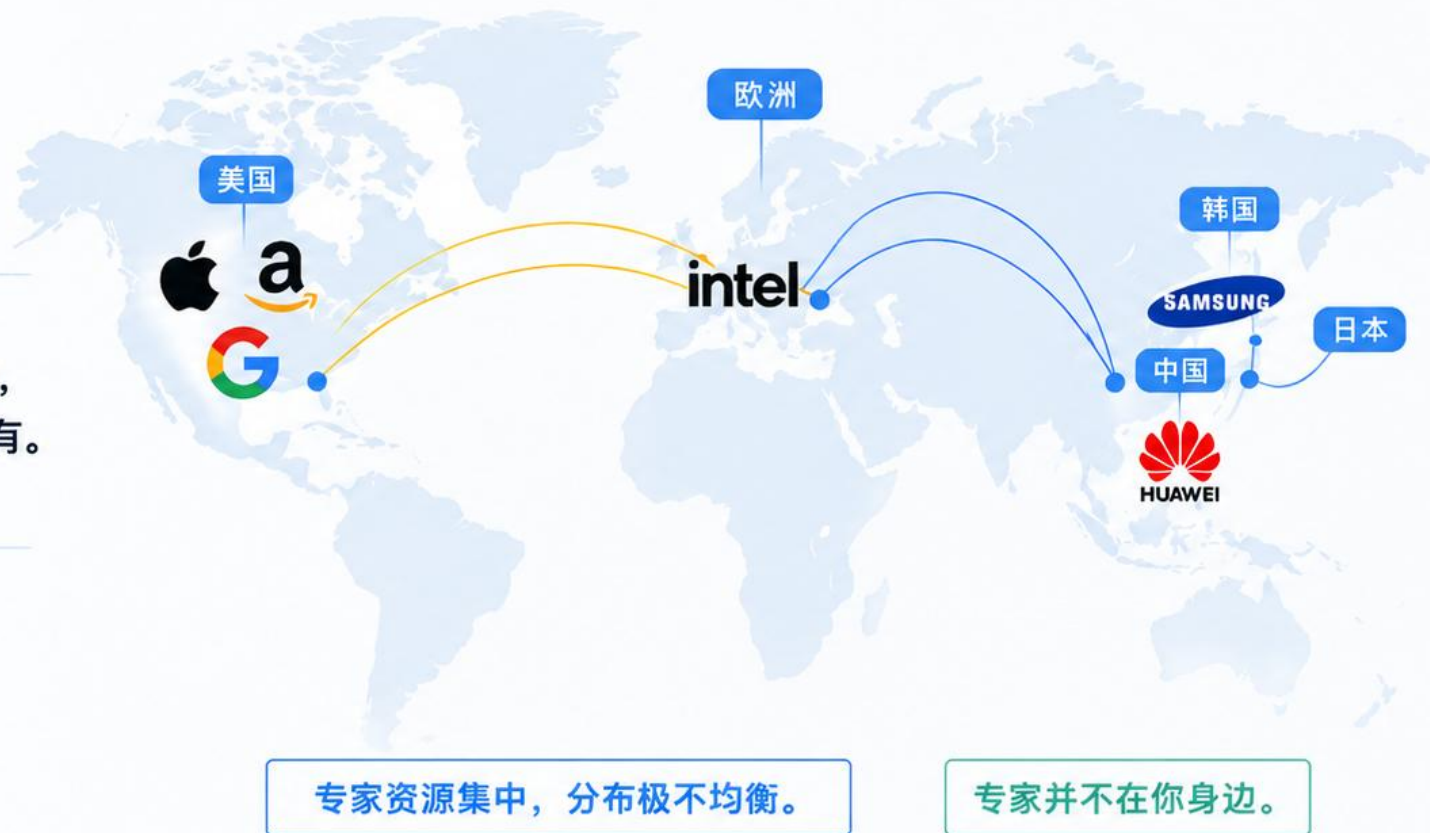
而大多数这些专家，
都集中在少数国家，
少数头部公司。



对于世界大多数地区的开发者来说，
你身边可能连一个这样的专家都没有。



今天，你所身处的地方，
依然决定了你能构建什么。



算子开发，依然处在“**手艺人时代**”——
这是一门只有少数人能参与的手艺活。

几天-几周
写一个算子

近百个算子
每个大模型

重新写
换一种芯片

模型越来越大，芯片越来越多，但写算子的人远远不够

AI写代码已经非常普及

- 大量AI辅助编程工具：Github Copilot, Claude, 通义灵码, 文心快码
- AI编程降低编程门槛：写代码 = 和AI对话

AI会写代码，但不会写“高性能代码”。

- 你让AI写一个网页，它可以很快写出来。
- 你让AI把算子优化到GPU极限性能，情况就完全不一样了：
 - 怎么用硬件
 - 怎么并行计算
 - 怎么减少内存访问

这不是语言问题，是工程问题。

而算子，恰好是**最难**的那一类工程。

AI coding的下一个阶段
正在发生在算子这一层

传统算子开发模式

- 阅读文献
- 理解算子公式
- 手写上百行代码
- 调试、调优



1

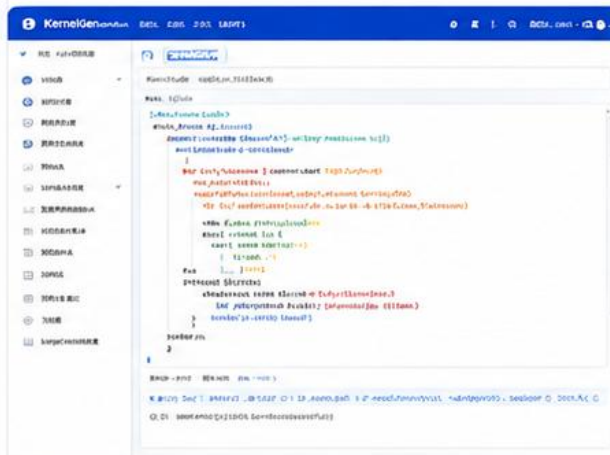
第一步

输入一句话需求



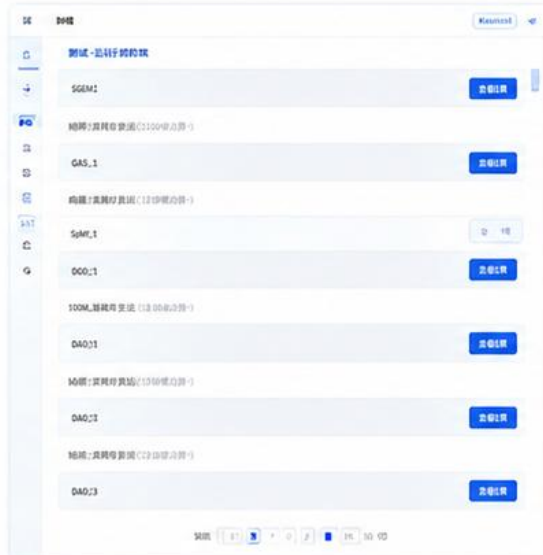
2

KernelGen自动生成GPU代码和参考实现



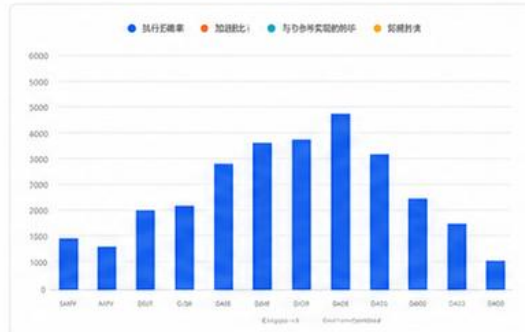
3

自动测试结果正确性和速度



4

可视化



查看包含正确性、加速比、不同数据规模表现的可视化完整结果

全过程十几分钟，实现从零基础到成功跑出GPU算子

一次编写，多芯运行

具备战略意义的重要能力

传统方式

为在芯片 B 上运行，
需要从头重写适配芯片 A 的内核



芯片 A

芯片 B

KernelGen 2.0

系统自动将您的内核迁移到不同芯片，
无需手动重写



自动迁移

国家发改委：指导国产大模型加大力度适配国产算力芯片

央视网 | 2026年05月22日 10:56:25

央视网消息：5月22日，国家发展改革委召开5月份新闻发布会。

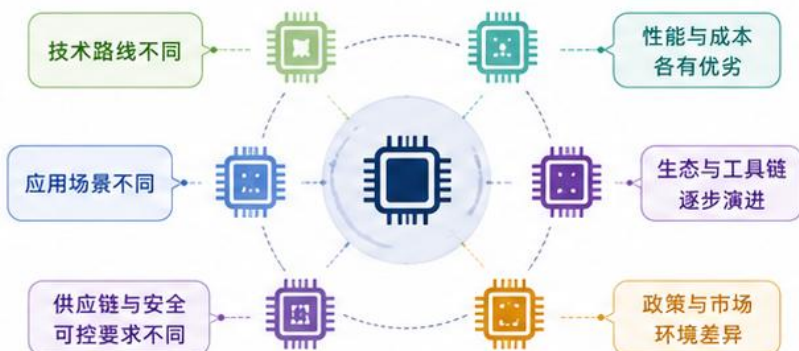
国家发展改革委政策研究室副主任、新闻发言人李超在新闻发布会上表示，人工智能领域核心技术和应用需求都呈现快速增长态势，我们始终坚持系统布局、分业施策、开放共享、安全可控，推动人工智能与经济社会各行业各领域广泛深度融合，指导国产大模型加大力度适配国产算力芯片，在保持快速发展的同时，确保自主可控、向善发展、行稳致远，让全体人民共享人工智能发展成果，这也是我国人工智能发展的一大突出特征。

未来不可能全世界只有 CUDA

未来中国 AI 的现实，很可能不是：“一种芯片统一世界” 而是：“长期多芯片共存”

所以：统一 AI 基础设施，会变成一个长期命题

多芯片共存是长期常态



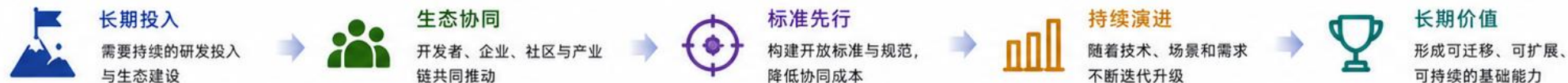
没有一种芯片能在所有维度上都最优

统一 AI 基础设施的核心目标



目标不是消灭多样性，而是在多样性之上实现统一、高效、可持续的 AI 开发与落地。

统一 AI 基础设施是一个长期建设工程







这对资源受限的团队来说 尤为重要。



在 NVIDIA H100 等
高端芯片上训练
一种算法可能需要
数十万美元。



有些团队可能
缺乏负担得起的
互联网访问权限，
从而难以获取
必要的硬件。



云资源通常
按小时租用，
成本会迅速
累积。

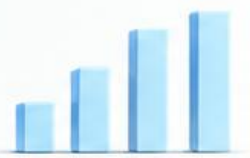


因此，软件层面的效率 比硬件本身更加重要。

如果你的内核足够高效，即使是普通芯片
也能产生出色的效果。



较差的内核优化



高度优化的内核



这正是 KernelGen 能做到的——
它帮助你在所拥有的芯片上，发挥出
尽可能好的性能。



最大化硬件
性能



降低成本，
节省资源



无需依赖大量
基础设施



用更少的资源
获得更好的成果

为什么通用 AI 编程写不了高性能算子，而 KernelGen 可以？

算子开发不是写代码，而是工程能力 + 领域经验 + 自动化闭环的综合系统工程

通用 AI 编程助手 (如 Copilot / Claude)

-   **缺乏领域知识 (从零猜)**
不了解算子结构、并行模式、内存访问等专业知识
-   **没有专家经验 (闭门造车)**
无法参考大量最优实现和优化技巧，容易走弯路
-   **优化能力弱 (试错式)**
靠反复修改和猜测，方向不稳定，难以达到极致性能
-   **人工参与多 (效率低)**
需要人反复指令、检查、修复、测试，周期长、成本高
-   **难以跨芯片迁移**
不了解不同芯片架构差异，很难一次适配多种硬件



结果：能写出来，但慢、贵、性能差、难迁移
像一个“实习生”，能完成任务，但不够专业

VS

KernelGen 2.0 (专家经验 + AI 自动化)

-   **拥有领域知识 (懂行)**
内置算子知识体系，理解算子结构、并行范式、性能关键点 **理解准确**
-   **专家经验加持 (站在巨人肩膀上)**
内置大量专家代码库和优化经验，参考最佳实践 **起点更高**
-   **自动优化 (系统化寻优)**
自动测量 → 分析瓶颈 → 自动改进 → 反复迭代直到达到性能目标 **性能更优**
-   **全自动闭环 (AI 自己把事情做完)**
生成 → 编译 → 测试 → 性能评估 → 自动优化 无需人工反复参与 **效率更高**
-   **一次开发，多芯片适配**
内置多芯片后端与适配策略，自动迁移和优化 **迁移更稳**



结果：写得对、跑得快、成本低、能迁移
像一个“资深专家团队”，专业、高效、可靠



本质区别：通用 AI 是“通才”，KernelGen 是“专才”。

写算子需要的不是会写代码，而是懂硬件、懂并行、懂优化、有经验、能自动化的工程能力。



不知疲倦的赛车调校师



自动迭代多轮，不需要人参与。

🔍 帮我把这个算子在摩尔线程上优化一下



一号选手



KernelGen 2.0

二号选手



纯AI编程助手

挑战项目：LayerNorm 算子
大模型每层调用的核心算子，性能优化的关键路径

评测指标	KernelGen	AI 编程助手
耗时	14 分钟	35 分钟
Token 消耗量	1万 tokens	2.2万 tokens
准确率	100%	100%

运行速度

1.14X

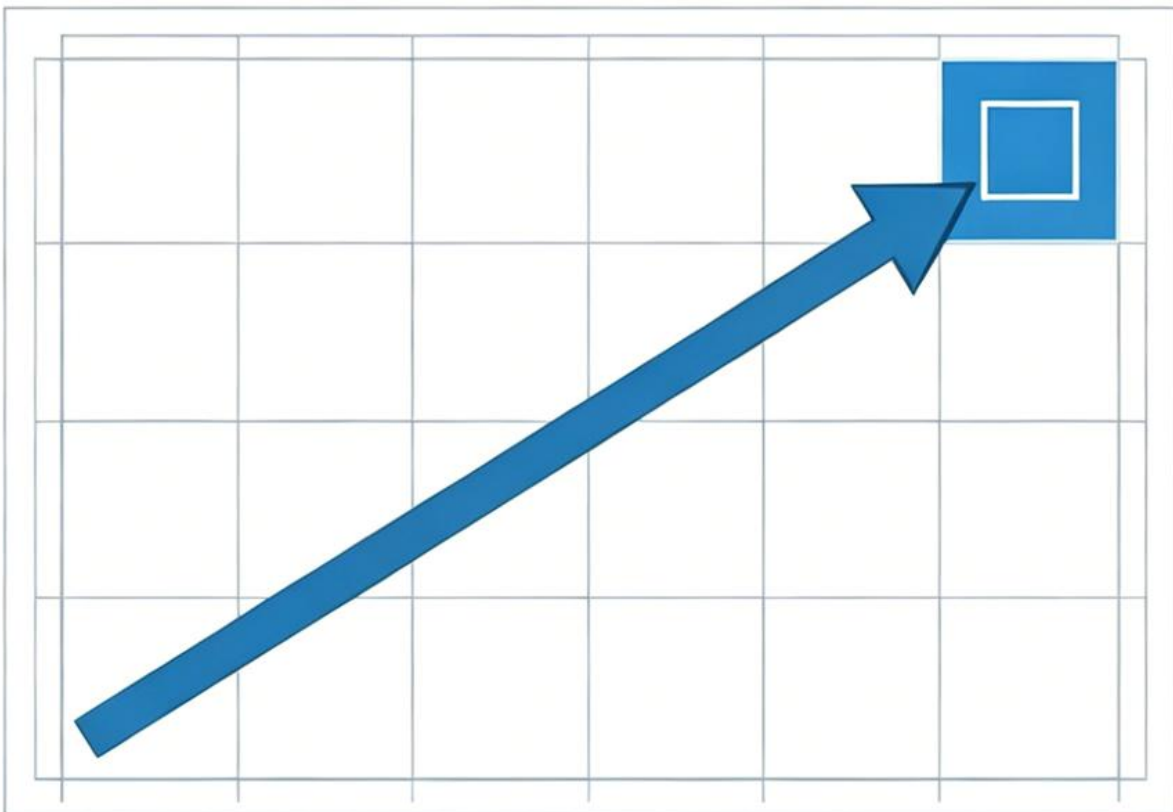
比标准实现更快

运行速度

0.64X

比标准实现慢

一半时间，一半资源，更快性能



KernelGen = 专家经验 × AI
站在专家肩膀上，方向明确

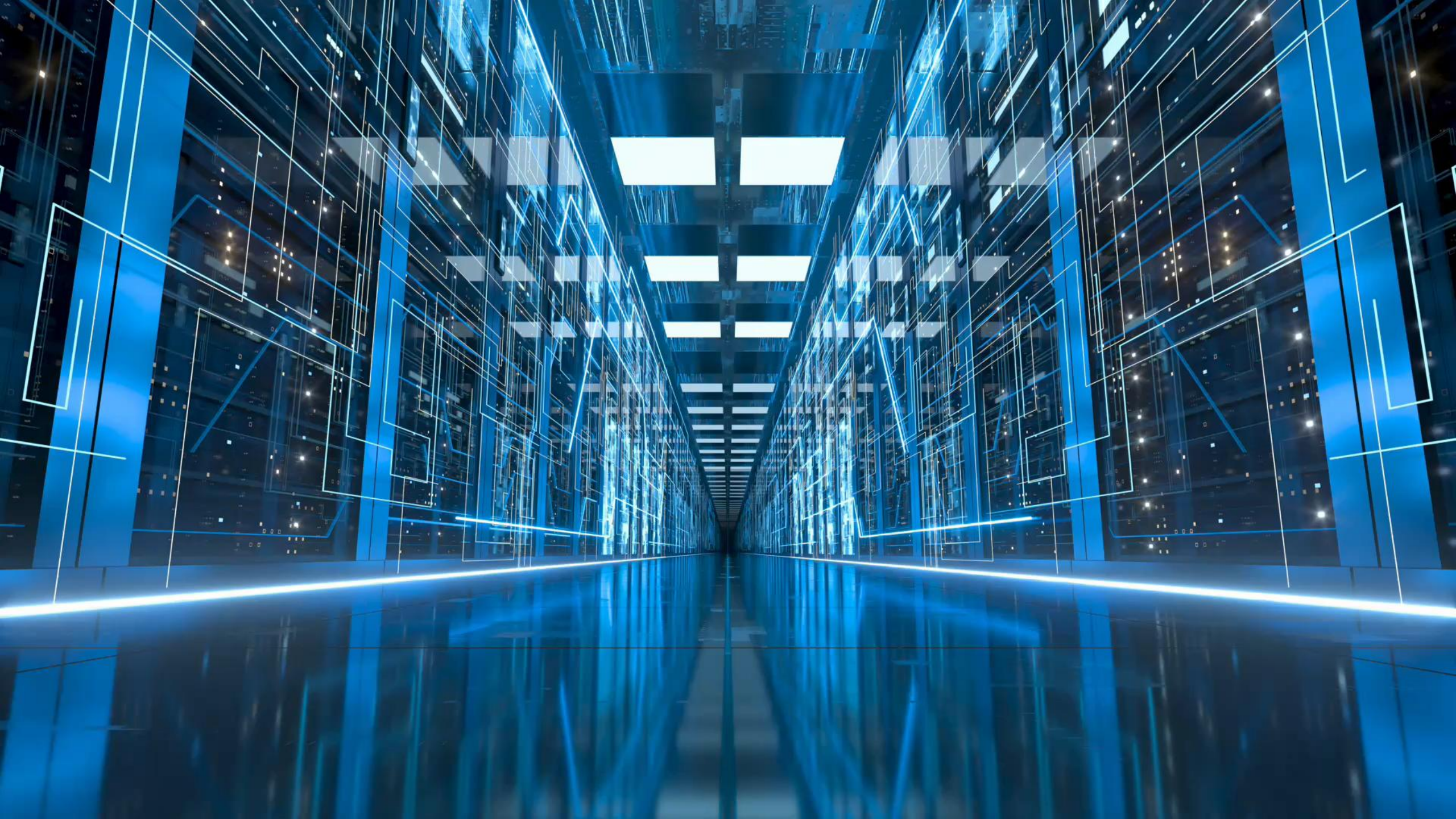


纯AI = AI + 试错
从零开始猜测，反复消耗资源

每次回答，背后是数万亿次计算在同时发生

北京智源人工智能研究院
BEIJING ACADEMY OF ARTIFICIAL INTELLIGENCE

用的是AI自己写的算子



谢谢！

KernelGen官网：<https://kernelgen.flagos.io/>

KernelGen Github: <https://github.com/flagos-ai/kernelgen>



KernelGen微信群